Midterm Report

Project Title

Automated Customer Service & Review Sentiment Analysis System

Project Goal

The primary goal of this project is to build an intelligent platform that significantly lowers customer service costs and accurately extracts product defects from raw consumer reviews.

Interdisciplinary Collaboration: This project deeply promotes interdisciplinary collaboration across AI, IT, engineering, and business fields:

- **Artificial Intelligence (AI):** We leverage cutting-edge Large Language Models (LLMs) combined with Retrieval-Augmented Generation (RAG) and Prompt Engineering (Few-shot, CoT) to achieve deep semantic understanding, sentiment classification, and natural language generation.
- **IT & Engineering:** The project requires robust software engineering practices, including building a responsive frontend (Streamlit), developing asynchronous high-concurrency backend APIs, deploying local vector databases, and ensuring the stability of the entire data pipeline.
- **Business:** The core driver of the technical implementation is commercial value. By quantifying customer emotions and summarizing product pain points (e.g., "damaged packaging"), the system delivers actionable business insights that guide product iteration, reduce return rates, and optimize supply chain decisions.

Business Idea

Small and medium-sized e-commerce sellers often struggle with slow responses to massive customer inquiries and lack the resources to systematically analyze voluminous user reviews. Our business concept is to provide an accessible, AI-driven SaaS solution. It features an **Intelligent Anthropomorphic Customer Representative** that mimics the merchant's exclusive tone and follows specific return/exchange protocols using a customized knowledge base. Simultaneously, its **Review Sentiment Analysis** engine transforms unstructured textual feedback into structured, visual business dashboards, helping merchants proactively iterate their products and minimize post-sales losses.

System Design

Hardware:

- Standard development PCs and servers for local training, script execution, and deployment handling.
- Cloud-based API infrastructure for LLM inference computation.

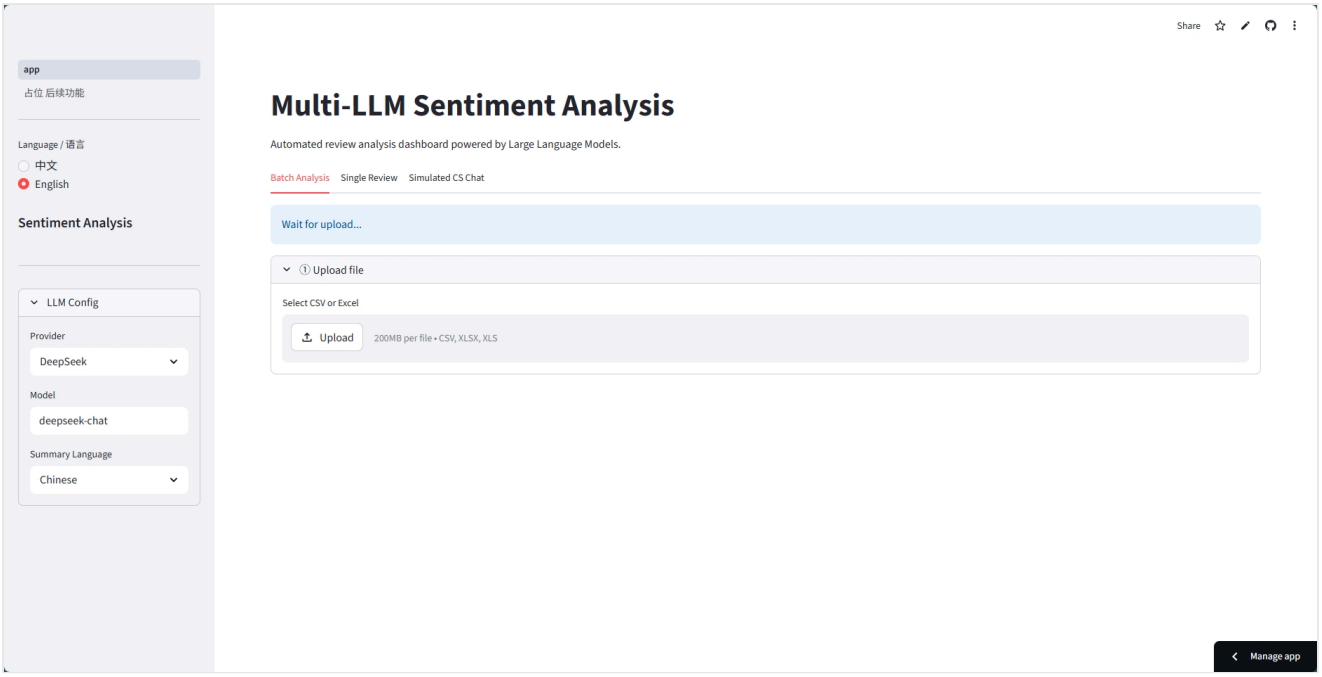
Software & Tech Stack:

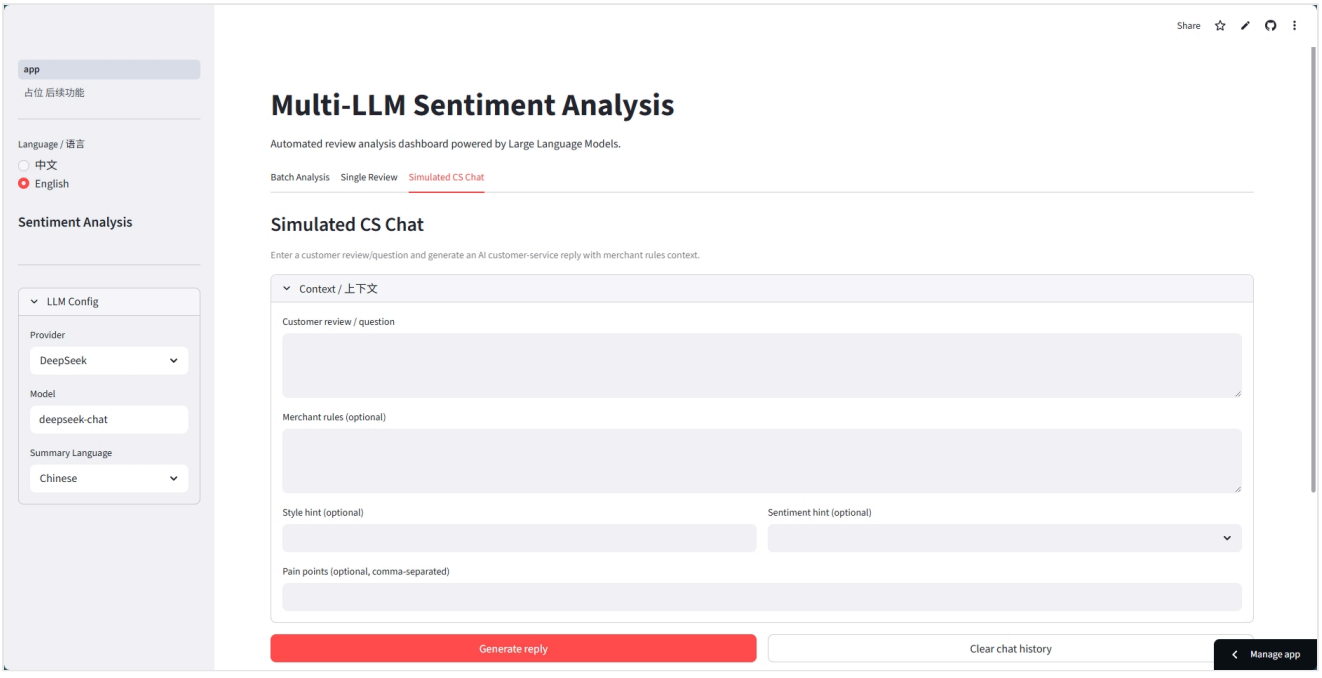
- **Frontend:** Python, Streamlit (for building interactive web applications and charts), Altair/Pandas (for data visualization and pre-processing).

- **Backend:** Python, `asyncio` / `httpx` for high-concurrency LLM API requests.
- **AI Core:** Google Gemini / DeepSeek LLM APIs.
- **Data & RAG:** LangChain / LlamaIndex framework, lightweight vector databases (e.g., ChromaDB or FAISS) for semantic chunking and merchant knowledge retrieval.

Prototype Images

Below is a demonstration of our current system prototype interfaces:





Future Plan (After Midterm)

Having successfully run the Minimum Viable Product (MVP) and passed the midterm milestone, our subsequent plan for Weeks 9-16 includes:

- **Testing & RAG Integration:** We will focus on architecting and integrating the scalable RAG Vector database, followed by finalizing quantitative A/B testing on RAG accuracy, and conducting rigorous End-to-End stress testing on our API pipelines. We will deliberately mine extreme edge-case data (e.g., sarcasm, emoji-only reviews) and deploy "Defensive Prompts" to prevent AI hallucinations.
- **Improvement:** Comprehensive backend refactoring, optimizing data synchronization mechanisms to solve concurrency bottlenecks, and perfecting the UI experience, such as adding a one-click export feature for AI-generated tables and visual analysis snapshots.
- **Final Demonstration Plan:** We will generate real-world "Business Insight Analysis Reports," implement a strict Feature Freeze, draft all required academic software documentation, prepare a highly polished final Powerpoint presentation, record a 3-5 minute backup demo video, and conduct "Red vs. Blue team" mock Q&A defenses to prepare for the final presentation.

Progress So Far

Overall Project Progress: 50% We have successfully completed Stage 1 and Stage 2 according to our Gantt chart. We have successfully mitigated the API rate limits, built a 200-item "Ground Truth" golden dataset, finalized the V2.0 Prompt architecture (achieving over 80% sentiment classification accuracy), successfully achieved merchant tone cloning through prompt-stuffing, and completed our interactive UI frontend mockups. We are currently transitioning into implementing the formal RAG knowledge base integration.

Individual Contributions

- **Backend Developer (LIU DEZHENG):** Fully responsible for designing the system's underlying architecture and LLM API interconnectivity. Successfully integrated mainstream API services (Google Gemini & DeepSeek), independently engineered asynchronous concurrency mechanisms (Token Bucket limiters & Retry) to efficiently handle thousands of data streams while bypassing Rate Limit bottlenecks, and is currently preparing to architect the scalable RAG vector database.
- **Frontend Developer (LIU JIACHENG):** Spearheaded the data visualization and user interaction infrastructure. Engineered robust preliminary data cleaning pipelines utilizing Pandas, and built an intuitive, interactive Web UI using the Streamlit framework. Implemented rich components including file upload parsing, dynamic data tables, and multi-dimensional statistical charts (pie charts, trend lines) to present business metrics intuitively.
- **Prompt Engineer & Tester (ZHENG WENBIN):** Focused on tuning and maximizing the core AI capabilities. Continuously iterated Prompt architectures using cutting-edge techniques (Few-shot, Chain of Thought), rigorously curated the golden validation datasets to track AI accuracy, and implemented systematic "Defensive Prompts" to isolate and mitigate AI hallucinations during extreme edge-case evaluations.

- **Data Collection (JI TENGFEI):** Managed the end-to-end data acquisition and annotation pipeline. Scraped authentic product review data from e-commerce channels, performed meticulous manual baseline annotation for the 200-item MVP Ground Truth testing set, and was entirely responsible for collecting, cleaning, and structuring the merchant's raw scripts into a hierarchical knowledge base standard for future RAG ingestion.